SUKKUR IBA UNIVERSITY
Department of Computer Science
Computer Architecture & Assembly Language

PROJECT PROPOSAL

Mini Compiler for AssemblyLang

A Complete Compiler Front-End for x86-Style Assembly Language

Group Leader	Muhammad Awais
Team Member	Azhar Ali
Program	BS Computer Science
Subject	Computer Architecture & Assembly Language
Supervisor	Dr. Hifazat Ali Shah
University	Sukkur IBA University
Submission Date	2026

TABLE OF CONTENTS

1. EXECUTIVE SUMMARY

This proposal describes the design and implementation of a Mini Compiler for AssemblyLang a simplified, x86-style assembly language created for educational purposes. The compiler covers all front-end phases: lexical analysis, recursive-descent parsing (LL(1)), semantic analysis with scope-aware symbol management, and Three-Address Code (TAC) generation. The project is delivered as a standalone Python 3 desktop application with an interactive Tkinter GUI, enabling students to write assembly code and observe every compilation phase in real time.

The work is submitted in partial fulfilment of the requirements of the Computer Architecture & Assembly Language course at Sukkur IBA University, under the supervision of Dr. Hifazat Ali Shah.

2. PROJECT OVERVIEW

2.1 Background

Compilers and assemblers are cornerstones of systems programming education. Understanding how source text is transformed token-by-token, rule-by-rule into executable intermediate code builds deep intuition for language design, computer architecture, and software correctness. A purpose-built mini compiler for an assembly-style language therefore serves as an ideal pedagogical tool.

2.2 Project Scope

The project implements a four-phase compiler front-end for AssemblyLang and wraps it in a five-tab graphical dashboard. The scope explicitly excludes back-end optimization and machine-code emission; the output artefact is Three-Address Code, which could be consumed by a separate code-generation back-end.

Language: AssemblyLang (custom x86-inspired)

Target IR: Three-Address Code (TAC)

Platform: Python 3 / Tkinter (cross-platform)

Team size: 1 developer (individual project)

3. PROBLEM STATEMENT

Teaching assembly language in isolation without accompanying compiler theory creates a gap: students can write assembly but cannot explain how a tool like NASM or MASM actually processes it. Standard compiler textbooks address high-level languages, leaving assembly-specific constructs (registers, memory references, size qualifiers, section directives, procedure bodies) largely unexplored in the compilation context. This project closes that gap by building a full compiler front-end purpose-designed for an assembly-style language.

Specific challenges addressed:

- Handling assembly-specific token classes: mnemonics, registers, size qualifiers, hex/decimal literals, and string literals within a single lexer pass.
- Designing a context-free grammar for assembly that is free of left recursion and ambiguity to enable deterministic LL(1) parsing.
- Managing two orthogonal scopes (DATA vs. CODE) plus nested procedure scopes inside a single symbol table.
- Mapping each assembly instruction to its canonical TAC equivalent (e.g., MOV becomes a copy through a fresh temporary; CMP is decomposed into a subtraction that sets a FLAGS temporary).

4. OBJECTIVES

4.1 Primary Objectives

- Implement a complete, error-tolerant lexer for AssemblyLang with precise line/column diagnostics.
- Construct an LL(1) recursive-descent parser that validates all grammatical rules defined in the CFG.
- Build a scope-aware symbol table that tracks DATA variables, CODE labels, and PROC definitions separately.
- Perform semantic analysis: duplicate-declaration detection, undeclared-identifier detection, and illegal-MOV-mem-mem detection.
- Generate Three-Address Code for every supported instruction, using a fresh temporary for each intermediate value.
- Deliver an interactive GUI dashboard that exposes tokens, symbol table, parse results, TAC output, and grammar documentation in five labelled tabs.

4.2 Secondary Objectives

- Provide a pre-loaded sample program covering all language features so the tool is immediately usable for in-class demonstration.
- Keep the codebase self-contained (standard library only) so installation is trivial on any Python 3 system.
- Structure the code so each compilation phase is independently readable and reusable.

5. LANGUAGE DEFINITION – ASSEMBLYLANG

5.1 Supported Instructions

Category	Mnemonics / Tokens
Data Transfer	MOV
Arithmetic	ADD, SUB, MUL, DIV
Comparison	CMP
Unconditional Jump	JMP
Conditional Jumps	JE, JNE, JG, JL, JGE, JLE
Stack Operations	PUSH, POP
Control Flow	CALL, RET, NOP, HLT
Procedure	PROC … ENDP
Section Directives	SECTION DATA, SECTION CODE
Size Directives	DB (1B), DW (2B), DD (4B), DQ (8B)
Size Qualifiers	BYTE PTR, WORD PTR, DWORD PTR, QWORD PTR
Registers	AX, BX, CX, DX, SP, BP, SI, DI + 8-bit halves AL/AH …

5.2 Grammar Transformations Applied

- **Left Recursion Removal:** StmtList → StmtList Stmt | Stmt transformed to StmtList → Stmt StmtList | ε
- **Left Factoring:** ID-prefixed rules factored to prevent non-determinism at the ID look-ahead.
- **Ambiguity Resolution:** Size-qualifier handling resolved before bracket parsing; MOV mem, mem flagged as a semantic (not syntactic) error.

6. SYSTEM ARCHITECTURE

6.1 Compilation Pipeline

Phase	Input / Output	Key Class
1 – Lexical	Source text → Token stream	Lexer
2 – Syntax	Token stream → Parse tree (implicit)	Parser (LL(1))
3 – Semantic	Parse tree → Annotated symbol table	Symbol Table
4 – Code Gen	Annotated tree → TAC instruction list	TACGenerator
GUI layer	All phases → Dashboard tabs	CompilerGUI

6.2 Symbol Table Scope Model

The Symbol Table class maintains a stack of dictionaries, one per scope level. Scope 0 is global; scope 1 is entered for SECTION DATA; scope 2 for SECTION CODE; scope 3+ for each PROC body. Lookup traverses from the innermost scope outward, matching standard lexical scoping rules.

7. IMPLEMENTATION DETAILS

7.1 Lexer

The Lexer class performs a single-pass, character-by-character scan. It supports:

- Decimal integer literals and hexadecimal literals in both 0x… prefix and …h suffix notation.
- Single- and double-quoted string literals with backslash escape sequences.
- Case-insensitive keyword and register recognition via an upper-case normalization pass.
- Comment skipping (semicolon to end-of-line) returning COMMENT tokens rather than discarding, enabling future IDE use.
- Precise line and column tracking for every token, feeding downstream error messages.

7.2 Parser

The Parser implements a hand-written LL(1) recursive-descent strategy. Every production rule in the CFG has a corresponding method. Key design points:

- The token stream is pre-filtered to remove NEWLINE and COMMENT tokens, simplifying all grammar methods.
- match() consumes the expected token type and records a syntax error (without raising an exception) if the token does not match, enabling error recovery.
- Semantic checks are embedded directly in parser methods: duplicate declaration on symbol_table.declare(), undeclared identifier on symbol_table.lookup(), and memory-operand conflict inside mov_instr().
- The parser builds the symbol table as a side-effect of parsing, so the TAC generator can reuse it without a separate pass.

7.3 TAC Generator

TACGenerator mirrors every parser method, consuming the same pre-filtered token stream with identical logic. This deliberate structural mirroring means that any grammar change propagates naturally.

TAC emission rules:

- MOV dst, src → t_n = src ; dst = t_n
- ADD/SUB/MUL/DIV dst, src → t_n = dst OP src ; dst = t_n
- CMP op1, op2 → t_n = op1 - op2 ; FLAGS = t_n
- JE/JNE/JG/JL/JGE/JLE lbl → if FLAGS <cond> goto lbl
- JMP lbl → goto lbl
- CALL/RET/PUSH/POP/NOP/HLT map to identically-named TAC pseudo-ops.

7.4 GUI Dashboard

CompilerGUI extends tk.Tk and organises the interface into:

- **Title bar:** project name in a dark-navy header.
- **Info bar:** university, course, developer and supervisor always visible.
- **Left panel:** source code editor with Compile, Clear, Sample, and About buttons.
- **Right panel (5 tabs):** Tokens table, Symbol Table, Parse Results log, TAC output, Grammar reference.
- **Status bar:** single-line compilation phase feedback.
- **Supervisor footer**: persistent display of supervisor and year.

8. TECHNOLOGY STACK

Component	Technology	Version / Notes
Programming Language	Python 3	3.8+ recommended
GUI Framework	Tkinter / ttk	Bundled with CPython
Data Structures	Python dicts	Symbol table scopes
Token Taxonomy	Python Enum	TokenType class
Intermediate Rep.	Plain text list	TAC string instructions
Dependency	None (stdlib)	Zero-install on Python 3

9. TIMELINE & MILESTONES

Phase	Deliverables / Tasks	Duration
Phase 1	Language specification; grammar design; left-recursion removal; left-factoring; first-set / follow-set computation	Week 1
Phase 2	Lexer implementation; token taxonomy (TokenType enum); lexical error handling; unit tests for all token classes	Week 1
Phase 3	LL(1) recursive-descent parser; all production rule methods; match() / error-recovery infrastructure	Week 1- 2
Phase 4	Symbol table with enter/exit scope; declare() / lookup(); DATA section parsing; PROC scope nesting	Week 2-3
Phase 5	Semantic analysis: duplicate declarations, undeclared identifiers, MOV mem-mem check; integration with parser	Week 3-4

Phase	Deliverables / Tasks	Duration
Phase 6	TACGenerator class mirroring parser; temporary allocation; TAC emission for all 19 instruction types	Week 3-4
Phase 7	Tkinter GUI: five-tab dashboard; compile pipeline wiring; sample program; grammar documentation tab	Week 4-5
Phase 8	Integration testing; edge-case error programs; documentation; final review; submission	Week 5-6

10. TESTING STRATEGY

10.1 Lexer Tests

- Valid and invalid decimal / hexadecimal literals.
- Unterminated string literals (should produce ERROR token + lexical error message).
- All keyword and register variations in mixed case.
- Comment stripping and newline handling.

10.2 Parser Tests

- Well-formed programs covering every production rule.
- Programs missing COMMA, RBRACKET, ENDP, etc. (expect syntax errors).
- Deeply nested memory references: DWORD PTR [BX+SI].

10.3 Semantic Tests

- Duplicate variable declarations in DATA section.
- Reference to undeclared identifier in operand position.
- MOV with two memory operands (should produce semantic error).
- CALL targeting a variable name rather than a PROC (should produce semantic error).

10.4 TAC Tests

- Each instruction type verified to emit the correct number and form of TAC lines.
- Temporary counter increments correctly across a multi-instruction program.
- Conditional jump table covers all seven jump mnemonics.

11. EXPECTED OUTCOMES

- A fully functional, single-file Python application (Assembly_Compiler.py) with zero external dependencies.
- A grammar document embedded in the GUI describing all production rules, transformations, and TAC mappings.
- A sample program exercising all 19 instruction types, both sections, memory references, and procedure calls.
- Complete error reporting for lexical, syntactic, and semantic errors, each pinpointed to line and column.
- An educational artefact suitable for live demonstration in a Computer Architecture lecture.

12. CONCLUSION

The Mini Compiler for AssemblyLang is a focused, well-scoped project that bridges two traditionally separate course Computer Architecture. By building a compiler front-end for an assembly-style language, the developer gains hands-on experience with every stage of the translation process: tokenization, parsing, symbol resolution, type checking, and intermediate code emission. The interactive GUI makes the compiler an effective teaching aid, allowing students to see exactly how each line of assembly is decomposed into tokens, validated against a formal grammar, recorded in a symbol table, and ultimately reduced to Three-Address Code.

The project is designed to be completed within a twelve-week semester, requires no external libraries, and is directly relevant to the learning outcomes of the Computer Architecture & Assembly Language course at Sukkur IBA University.

Submitted by: Muhammad Awais | Supervised by: Dr. Hifazat Ali Shah | Sukkur IBA University | 2026